

In [62]: `import pandas as pd`

In [63]: `df1 = pd.read_excel('C:/Users/USER/OneDrive/Documents/HNG_Internship_Git/Stage_8/St
df2 = pd.read_excel('C:/Users/USER/OneDrive/Documents/HNG_Internship_Git/Stage_8/St
df3 = pd.read_excel('C:/Users/USER/OneDrive/Documents/HNG_Internship_Git/Stage_8/St
df4 = pd.read_excel('C:/Users/USER/OneDrive/Documents/HNG_Internship_Git/Stage_8/St
df5 = pd.read_excel('C:/Users/USER/OneDrive/Documents/HNG_Internship_Git/Stage_8/St`

In [64]: `df1.shape, df2.shape, df3.shape, df4.shape, df5.shape`

Out[64]: ((179, 4), (119, 3), (80, 6), (77, 6), (36, 4))

In [69]: `df1.head()`

Out[69]:

	Full Name	Slack ID	Stage1_Grade	Stage1_Decision
0	Precious Sampson	@PreciousS	9.00	Pass
1	Precious Daniel	@Precious Daniel Sunday	9.00	Pass
2	Adekemi Adetulubo	@JEWAL	8.75	Pass
3	Blessing Motue	BlessingM	8.75	Pass
4	Adekunle Faderera	@Agnes A	8.50	Pass

In [66]: `df1.columns = ['First Name', 'Slack ID', 'Stage1_Grade', 'Stage1_Decision',]`

In [68]: `df1.rename(columns={
 'First Name' : 'Full Name',
}, inplace=True)`

In [70]: `df1.head()`

Out[70]:

	Full Name	Slack ID	Stage1_Grade	Stage1_Decision
0	Precious Sampson	@PreciousS	9.00	Pass
1	Precious Daniel	@Precious Daniel Sunday	9.00	Pass
2	Adekemi Adetulubo	@JEWAL	8.75	Pass
3	Blessing Motue	BlessingM	8.75	Pass
4	Adekunle Faderera	@Agnes A	8.50	Pass

In [73]: `df2.head()`

Out[73]:

	Full Name	Stage2_Grade	Stage2_Decision
0	Chidubem Ojukwu	9.6	Pass
1	Precious Sampson	9.25	Pass
2	Tolulope Akinroye	9.25	Pass
3	Rhoda Anioke	9.25	Pass
4	Adekemi Adetulubo	9	Pass

In [72]: `df2.columns = ['Full Name', 'Stage2_Grade', 'Stage2_Decision',]`In [76]: `df3.head()`

Out[76]:

	Full Name	Slack ID	email	Stage3_Grade	remark	Sta
0	Esther Omah	AirStar Visuals	eniola.omah@gmail.com	9.0	NaN	
1	OgheneFego Umuteme	FegoU	umutemefego@gmail.com	9.0	Outsanding documentations!	
2	Victoria Adebayo	@Vikkyjay	taiwoadebayo093@gmail.com	8.5	NaN	
3	Awele Uzoechina	@AweleB	awelezubugor1@gmail.com	8.5	NaN	
4	Ugwu, Uchechi	@JulianaU	uchechijuliana5@gmail.com	8.5	NaN	

In [75]: `df3.columns = ['Full Name', 'Slack ID', 'email', 'Stage3_Grade', 'remark', 'Stage3_`In [79]: `df4.head()`

Out [79]:

	Full Name	Slack ID	email	Stage4_Grade	Stage
0	Adekemi Adetulubo	JEWAL	adekemitulubo@gmail.com	10.0	
1	Grace Ochinokwu	@Enohmaje	dataanalystenoh@gmail.com	9.0	
2	Lorine Oboh	@Lorine Oboh	lorineebosetale@gmail.com	9.0	
3	Martha Omolabi	@random_entity	marthaomolabi@gmail.com	8.5	
4	KEHINDE Blessing Temitope	@Blessing.T.	Kehindeblessingtemitope2001@gmail.com	8.0	

In [78]: `df4.columns = ['Full Name', 'Slack ID', 'email', 'Stage4_Grade', 'Stage4_Decision',`In [82]: `df5.head()`

Out [82]:

	Full Name	Slack ID	Stage5_Grade	Stage3_Decision
0	Oluebube Egesi	E. Oluebube	8.5	Pass
1	Chidubem Ojukwu	@COTworks	8.0	Pass
2	Jackson Elizabeth. O	@Lisa_J	7.5	Pass
3	Adekemi Adetulubo	@JEWAL	7.5	Pass
4	KEHINDE Blessing Temitope	@Blessing.T.	7.0	Pass

In [81]: `df5.columns = ['Full Name', 'Slack ID', 'Stage5_Grade', 'Stage3_Decision',]`
In [83]: `new_df = df1.merge(df2, on=['Full Name'], how='left')\
 .merge(df3, on=['Full Name', 'Slack ID'], how='outer')\
 .merge(df4, on=['Full Name', 'Slack ID'], how='outer')\
 .merge(df5, on=['Full Name', 'Slack ID'], how='outer')`
In [84]: `new_df.head()`

Out[84]:

	Full Name	Slack ID	Stage1_Grade	Stage1_Decision	Stage2_Grade	Stage2_Decision
0	Juliet Madueke	@JulietM	NaN	NaN	NaN	NaN
1	Ruth Adeniji	@Adeayo - FTA's VA	8.00	Pass	NaN	NaN
2	Aanuoluwapo Raheem	AnuT	4.25	Fail	NaN	NaN
3	Adedigba Kehinde	@Keneliz	7.50	Pass	NaN	NaN
4	Adedigba Kehinde	@keneliz	NaN	NaN	NaN	NaN

In [85]: `new_df.isnull().sum()`

```
Out[85]: Full Name      0
Slack ID      0
Stage1_Grade  128
Stage1_Decision 128
Stage2_Grade  269
Stage2_Decision 269
email_x      228
Stage3_Grade  228
remark       295
Stage3_Decision_x 228
email_y      232
Stage4_Grade  232
Stage4_Decision 232
comments     288
Stage5_Grade  273
Stage3_Decision_y 273
dtype: int64
```

In [86]: `new_df.sort_values("Full Name")`

Out[86]:

	Full Name	Slack ID	Stage1_Grade	Stage1_Decision	Stage2_Grade	Stage2_I
0	Juliet Madueke	@JulietM	NaN	NaN	NaN	
1	Ruth Adeniji	@Adeayo - FTA's VA	8.00	Pass	NaN	
2	Aanuoluwapo Raheem	AnuT	4.25	Fail	NaN	
3	Adedigba Kehinde	@Keneliz	7.50	Pass	NaN	
4	Adedigba Kehinde	@keneliz	NaN	NaN	NaN	
...	...	...	...	...	...	...
304	Wonders Emmanuel	WondersE	NaN	NaN	NaN	
305	Wonders Emmanuel	@WondersE	NaN	NaN	NaN	
306	Wonders Emmanuel	WondersE	NaN	NaN	NaN	
307	Yekini Samson	@Samsyne	4.00	Fail	NaN	
308	chinenye chikezie	Chi_TechSavvyVA	NaN	NaN	NaN	

309 rows × 16 columns



```
In [87]: def Full_Name(name):
    if pd.isna(name):
        return ""
    name = str(name).strip() # Strip leading/trailing whitespace
    # Remove @ symbol and any surrounding spaces
    name = re.sub(r'^@+|@+$', '', name).strip()
    # Remove emojis/suffixes like "1", "2"
    name = re.sub(r'[0-9]|\s*\d+\s*', '', name).strip()
    # Normalize spaces
    name = ' '.join(name.split())
    return name.lower()
```

```
In [98]: def clean_full_name(name):
    if pd.isna(name):
        return ""
    return str(name).strip().title() # or .lower(), depending on your need

# Then apply it
new_df['Full_Name'] = new_df['Full Name'].apply(clean_full_name)
```

In [103...

new_df.head()

Out[103...

	Full Name	Slack ID	Stage1_Grade	Stage1_Decision	Stage2_Grade	Stage2_Decision
0	Juliet Madueke	@JulietM	NaN	NaN	NaN	NaN
1	Ruth Adeniji	@Adeayo - FTA's VA	8.00	Pass	NaN	NaN
2	Aanuoluwapo Raheem	AnuT	4.25	Fail	NaN	NaN
3	Adedigba Kehinde	@Keneliz	7.50	Pass	NaN	NaN
4	Adedigba Kehinde	@keneliz	NaN	NaN	NaN	NaN

In [102...

```
new_df['Full Name'] = new_df['Full_Name']
new_df.drop(columns=['Full_Name'], inplace=True)
```

In [106...

```
# Define columns to preserve (all except helper column)
cols_to_keep = [col for col in new_df.columns if col != 'Name_Clean']

# Group by cleaned name, aggregate: take first non-null value
clean_df = new_df.groupby('Full Name', as_index=False).agg(
    lambda x: x.dropna().iloc[0] if not x.dropna().empty else None
)

# Drop the helper column
clean_df = clean_df.drop(columns=['Full Name'])

# Reset index
clean_df = clean_df.reset_index(drop=True)

print(f"Original rows: {len(new_df)}")
print(f"Unique people (after cleaning): {len(clean_df)}")
```

Original rows: 309

Unique people (after cleaning): 214

In [108...

```
print(new_df[['Full Name']].head(10))
```

```

Full Name
0 Juliet Madueke
1 Ruth Adeniji
2 Aanuoluwapo Raheem
3 Adedigba Kehinde
4 Adedigba Kehinde
5 Adekemi Adetulubo
6 Adekemi Adetulubo
7 Adekemi Adetulubo
8 Adekunle Faderera
9 Adeola Testimony

```

In [118... `new_df.head()`

Out[118...

	Full Name	Slack ID	Stage1_Grade	Stage1_Decision	Stage2_Grade	Stage2_Decision
0	Juliet Madueke	@JulietM	NaN	NaN	NaN	NaN
1	Ruth Adeniji	@Adeayo - FTA's VA	8.00	Pass	NaN	NaN
2	Aanuoluwapo Raheem	AnuT	4.25	Fail	NaN	NaN
3	Adedigba Kehinde	@Keneliz	7.50	Pass	NaN	NaN
4	Adedigba Kehinde	@keneliz	NaN	NaN	NaN	NaN

5 rows × 21 columns



In [133... `print(f"Original data shape: {new_df.shape}")`
`print(f"Number of unique names: {new_df['Full Name'].nunique()}")`
`print(f"\nDuplicate names:")`
`print(new_df[new_df.duplicated('Full Name', keep=False)][['Full Name']].value_counts()`

Original data shape: (309, 21)

Number of unique names: 214

Duplicate names:

Full Name

Chinaza Uwa 4

Chinenye Chikezie 4

Esther Omah 4

Mercy Igbokwe 4

Wonders Emmanuel 4

..

Queenesther Ubah 2

Uchechi Osuagwu 2

Tolulope Akinroye 2

Rofiat Ileyemi 2

Victor Omotowoju 2

Name: count, Length: 62, dtype: int64

```
In [134... agg_dict = {
    'Slack ID': 'first',
    'Stage1_Grade': 'max',
    'Stage1_Decision': 'first',
    'Stage2_Grade': 'max',
    'Stage2_Decision': 'first',
    'email_x': 'first',
    'Stage3_Grade': 'max',
    'remark': lambda x: '; '.join(x.dropna().astype(str).unique()) if x.notna().any
    'Stage3_Decision_x': 'first',
    'email_y': 'first',
    'Stage4_Grade': 'max',
    'Stage4_Decision': 'first',
    'comments': lambda x: '; '.join(x.dropna().astype(str).unique()) if x.notna().a
    'Stage5_Grade': 'max',
    'Stage3_Decision_y': 'first',
    'Stage1_perf_grade': 'first',
    'Stage2_perf_grade': 'first',
    'Stage3_perf_grade': 'first',
    'Stage4_perf_grade': 'first',
    'Stage5_perf_grade': 'first'
}
```

```
In [136... agg_dict_filtered = {k: v for k, v in agg_dict.items() if k in new_df.columns}
```

```
In [139... import numpy as np
consolidated_df = new_df.groupby('Full Name', as_index=False).agg(agg_dict_filtered
```

```
In [140... print(f"\n\nConsolidated data shape: {consolidated_df.shape}")
print(f"\nFirst few rows of consolidated data:")
print(consolidated_df.head(10))
```


Consolidated data shape: (214, 21)

First few rows of consolidated data:

	Full Name	Slack ID \
0	Aanuoluwapo Raheem	AnuT
1	Adedigba Kehinde	@Keneliz
2	Adekemi Adetulubo	@JEWAL
3	Adekunle Faderera	@Agnes A
4	Adeola Testimony	Adetola Testimony Oluwafikunayomi
5	Adetola Testimony Oluwafikunayomi	@Adetola Testimony Oluwafikunayomi
6	Adetona Apelehin	AdetolaA
7	Adeyemi Adedolapo Kavan	kavanoa1
8	Adeyemi Zainab	@AZA
9	Afe Tolulope	Simply Tee

	Stage1_Grade	Stage1_Decision	Stage2_Grade	Stage2_Decision \
0	4.25	Fail	NaN	None
1	7.50	Pass	NaN	None
2	8.75	Pass	9.0	Pass
3	8.50	Pass	NaN	None
4	8.00	Pass	NaN	None
5	NaN	None	NaN	None
6	3.25	Fail	NaN	None
7	6.00	Pass	NaN	None
8	6.50	Pass	NaN	None
9	6.50	Pass	NaN	None

	email_x	Stage3_Grade	remark	Stage3_Decision_x	...	\
0	None	NaN	NaN	None	...	
1	Adedigbakehinde1002gmail.com	5.0	NaN	fail	...	
2	adekemitulubo@gmail.com	6.0	NaN	pass	...	
3	None	NaN	NaN	None	...	
4	None	NaN	NaN	None	...	
5	adetolatestimony7@gmail.com	7.5	NaN	pass	...	
6	None	NaN	NaN	None	...	
7	None	NaN	NaN	None	...	
8	None	NaN	NaN	None	...	
9	None	NaN	NaN	None	...	

	Stage4_Grade	Stage4_Decision	comments	Stage5_Grade	Stage3_Decision_y	\
0	NaN	None	NaN	NaN	None	
1	7.0	Pass	NaN	NaN	None	
2	10.0	Pass	NaN	7.5	Pass	
3	NaN	None	NaN	NaN	None	
4	NaN	None	NaN	NaN	None	
5	NaN	None	NaN	NaN	None	
6	NaN	None	NaN	NaN	None	
7	NaN	None	NaN	NaN	None	
8	NaN	None	NaN	NaN	None	
9	NaN	None	NaN	NaN	None	

	Stage1_perf_grade	Stage2_perf_grade	Stage3_perf_grade	Stage4_perf_grade \
0	Fail	No Score	No Score	No Score
1	Second Class Upper	No Score	No Score	No Score
2	Distinction	Distinction	Third Class	No Score

3	Distinction	No Score	No Score	No Score
4	Distinction	No Score	No Score	No Score
5	No Score	No Score	Second Class Upper	No Score
6	Fail	No Score	No Score	No Score
7	Third Class	No Score	No Score	No Score
8	Third Class	No Score	No Score	No Score
9	Third Class	No Score	No Score	No Score

	Stage5_perf_grade
0	No Score
1	No Score
2	No Score
3	No Score
4	No Score
5	No Score
6	No Score
7	No Score
8	No Score
9	No Score

[10 rows x 21 columns]

```
In [143...] consolidated_df.to_csv('consolidated_new_df.csv', index=False)
print("\n✓ Consolidated data saved to 'consolidated_new_df.csv'")
```

✓ Consolidated data saved to 'consolidated_new_df.csv'

```
In [158...] new_df = pd.read_excel('C:/Users/USER/OneDrive/Documents/HNG_Internship_Git/Stage_8
```

```
In [159...] def score_grade(Grade):
    if pd.isnull(Grade):
        return 'No Score'
    elif Grade >= 8:
        return 'Distinction'
    elif Grade >= 7:
        return 'Second Class'
    elif Grade >= 6:
        return 'Third Class'
    elif Grade >= 5:
        return 'Let my people go'
    else:
        return 'Fail'
```

```
In [160...] new_df["Stage1_perf_grade"] = new_df["Stage1_Grade"].apply(score_grade)
new_df["Stage2_perf_grade"] = new_df["Stage2_Grade"].apply(score_grade)
new_df["Stage3_perf_grade"] = new_df["Stage3_Grade"].apply(score_grade)
new_df["Stage4_perf_grade"] = new_df["Stage4_Grade"].apply(score_grade)
new_df["Stage7_perf_grade"] = new_df["Stage7_Grade"].apply(score_grade)
```

```
In [162...] # new_df["Stage1_perf_grade"].value_counts()
new_df["Stage2_perf_grade"].value_counts()
# new_df["Stage3_perf_grade"].value_counts()
# new_df["Stage4_perf_grade"].value_counts()
# new_df["Stage7_perf_grade"].value_counts()
```

```
Out[162...] Stage2_perf_grade
No Score      158
Distinction   11
Third Class   11
Second Class   9
Let my people go 6
Fail          1
Name: count, dtype: int64
```

```
In [145...] import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder
import joblib
import warnings
warnings.filterwarnings('ignore')
```

```
In [146...] new_df = pd.read_csv('C:/Users/USER/OneDrive/Documents/HNG_Internship_Git/Stage_8/S
print(f"Original data shape: {new_df.shape}")
print(f"Number of unique names: {new_df['Full Name'].nunique()}")
```

Original data shape: (196, 17)

Number of unique names: 196

```
In [147...] duplicates = new_df[new_df.duplicated('Full Name', keep=False)]['Full Name'].value_
if len(duplicates) > 0:
    print(f"\nDuplicate names found: {len(duplicates)}")
    print(duplicates.head())
```

```
In [148...] agg_dict = {
    'Slack ID': 'first',
    'Stage1_Grade': 'max',
    'Stage1_Decision': 'first',
    'Stage2_Grade': 'max',
    'Stage2_Decision': 'first',
    'Stage3_Grade': 'max',
    'Stage3_Decision': 'first',
    'Stage4_Grade': 'max',
    'Stage4_Decision': 'first',
    'Stage7_Grade': 'max',
    'Stage7_Decision': 'first',
    'Stage1_perf_grade': 'first',
    'Stage2_perf_grade': 'first',
    'Stage3_perf_grade': 'first',
    'Stage4_perf_grade': 'first',
    'Stage7_perf_grade': 'first'
}
```

```
In [149...] agg_dict_filtered = {k: v for k, v in agg_dict.items() if k in new_df.columns}
new_df = new_df.groupby('Full Name', as_index=False).agg(agg_dict_filtered)
print(f"After consolidation: {new_df.shape}")
```

After consolidation: (196, 17)

In [150...

```

stages_data = []

for _, row in new_df.iterrows():
    # Stage 1
    if pd.notna(row['Stage1_Grade']):
        stages_data.append({
            'Full Name': row['Full Name'],
            'Slack ID': row['Slack ID'],
            'Stage': 'Stage 1',
            'Stage_Number': 1,
            'Grade': row['Stage1_Grade'],
            'Decision': row['Stage1_Decision'],
            'Perf_Grade': row['Stage1_perf_grade']
        })

    # Stage 2
    if pd.notna(row['Stage2_Grade']):
        stages_data.append({
            'Full Name': row['Full Name'],
            'Slack ID': row['Slack ID'],
            'Stage': 'Stage 2',
            'Stage_Number': 2,
            'Grade': row['Stage2_Grade'],
            'Decision': row['Stage2_Decision'],
            'Perf_Grade': row['Stage2_perf_grade']
        })

    # Stage 3
    if pd.notna(row['Stage3_Grade']):
        stages_data.append({
            'Full Name': row['Full Name'],
            'Slack ID': row['Slack ID'],
            'Stage': 'Stage 3',
            'Stage_Number': 3,
            'Grade': row['Stage3_Grade'],
            'Decision': row['Stage3_Decision'],
            'Perf_Grade': row['Stage3_perf_grade']
        })

    # Stage 4
    if pd.notna(row['Stage4_Grade']):
        stages_data.append({
            'Full Name': row['Full Name'],
            'Slack ID': row['Slack ID'],
            'Stage': 'Stage 4',
            'Stage_Number': 4,
            'Grade': row['Stage4_Grade'],
            'Decision': row['Stage4_Decision'],
            'Perf_Grade': row['Stage4_perf_grade']
        })

    # Stage 7
    if pd.notna(row['Stage7_Grade']):
        stages_data.append({
            'Full Name': row['Full Name'],

```

```

'Slack ID': row['Slack ID'],
'Stage': 'Stage 7',
'Stage_Number': 7,
'Grade': row['Stage7_Grade'],
'Decision': row['Stage7_Decision'],
'Perf_Grade': row['Stage7_perf_grade']
})

```

```

In [152... new_df_long = pd.DataFrame(stages_data)
print(f"Long format shape: {new_df_long.shape}")
print(f"\nSample of long format data:")
print(new_df_long.head(10))

```

Long format shape: (408, 7)

Sample of long format data:

	Full Name	Slack ID	Stage	Stage_Number	Grade	Decision	\
0	Aanuoluwapo Raheem	AnuT	Stage 1	1	4.25	Fail	
1	Adedigba Kehinde	@Keneliz	Stage 1	1	7.50	Pass	
2	Adedigba Kehinde	@Keneliz	Stage 3	3	5.00	fail	
3	Adedigba Kehinde	@Keneliz	Stage 4	4	7.00	Pass	
4	Adekemi Adetulubo	@JEWAL	Stage 1	1	8.75	Pass	
5	Adekemi Adetulubo	@JEWAL	Stage 2	2	9.00	Pass	
6	Adekemi Adetulubo	@JEWAL	Stage 3	3	6.00	pass	
7	Adekemi Adetulubo	@JEWAL	Stage 4	4	10.00	Pass	
8	Adekemi Adetulubo	@JEWAL	Stage 7	7	7.50	Pass	
9	Adekunle Faderera	@Agnes A	Stage 1	1	8.50	Pass	

	Perf_Grade
0	Fail
1	Second Class
2	Let my people go
3	Second Class
4	Distinction
5	Distinction
6	Third Class
7	Distinction
8	Second Class
9	Distinction

```

In [153... new_df_long.head()

```

Out[153...

	Full Name	Slack ID	Stage	Stage_Number	Grade	Decision	Perf_Grade
0	Aanuoluwapo Raheem	AnuT	Stage 1	1	4.25	Fail	Fail
1	Adedigba Kehinde	@Keneliz	Stage 1	1	7.50	Pass	Second Class
2	Adedigba Kehinde	@Keneliz	Stage 3	3	5.00	fail	Let my people go
3	Adedigba Kehinde	@Keneliz	Stage 4	4	7.00	Pass	Second Class
4	Adekemi Adetulubo	@JEWAL	Stage 1	1	8.75	Pass	Distinction

In [157...

```
new_df_long.to_csv('stages_long_format.csv', index=False)
```

In [158...

```
grade_cols = ['Stage1_Grade', 'Stage2_Grade', 'Stage3_Grade', 'Stage4_Grade', 'Stage5_Grade']
new_df[grade_cols] = new_df[grade_cols].fillna(0)
```

In [159...

```
new_df_long.head()
```

Out[159...

	Full Name	Slack ID	Stage	Stage_Number	Grade	Decision	Perf_Grade
0	Aanuoluwapo Raheem	AnuT	Stage 1	1	4.25	Fail	Fail
1	Adedigba Kehinde	@Keneliz	Stage 1	1	7.50	Pass	Second Class
2	Adedigba Kehinde	@Keneliz	Stage 3	3	5.00	fail	Let my people go
3	Adedigba Kehinde	@Keneliz	Stage 4	4	7.00	Pass	Second Class
4	Adekemi Adetulubo	@JEWAL	Stage 1	1	8.75	Pass	Distinction

In [163...

```
# Create features
new_df['average_grade'] = new_df[grade_cols].mean(axis=1)
new_df['grade_improvement'] = new_df['Stage7_Grade'] - new_df['Stage1_Grade']
new_df['max_grade'] = new_df[grade_cols].max(axis=1)
new_df['min_grade'] = new_df[grade_cols].min(axis=1)
new_df['grade_std'] = new_df[grade_cols].std(axis=1)
new_df['completion_rate'] = new_df[grade_cols].apply(lambda x: (x > 0).sum() / len(x))
new_df['consistent_performer'] = new_df[grade_cols].apply(lambda x: x.std() < 1, axis=1)
new_df['passed_all_stages'] = new_df[grade_cols].apply(lambda x: all(g >= 5 for g in x))
```

In [164...

```
print(f"\nNew features created:")
print(new_df[['Full Name', 'average_grade', 'grade_improvement', 'max_grade', 'min_grade', 'grade_std', 'completion_rate', 'consistent_performer', 'passed_all_stages']])
```

New features created:

	Full Name	average_grade	grade_improvement	\
0	Aanuoluwapo Raheem	0.85	-4.25	
1	Adedigba Kehinde	3.90	-7.50	
2	Adekemi Adetulubo	8.25	-1.25	
3	Adekunle Faderera	1.70	-8.50	
4	Adetola Testimony Oluwafikunayomi	3.10	-8.00	

	max_grade	min_grade	grade_std	completion_rate	consistent_performer	\
0	4.25	0.0	1.900658	0.2	0	
1	7.50	0.0	3.681032	0.6	0	
2	10.00	6.0	1.541104	1.0	0	
3	8.50	0.0	3.801316	0.2	0	
4	8.00	0.0	4.248529	0.4	0	

	passed_all_stages
0	0
1	0
2	1
3	0
4	0

In [165... new_df.head()

Out[165...

	Full Name	Slack ID	Stage1_Grade	Stage1_Decision	Stage2_Grade	Stage2
0	Aanuoluwapo Raheem	AnuT	4.25	Fail	0.0	
1	Adedigba Kehinde	@Keneliz	7.50	Pass	0.0	
2	Adekemi Adetulubo	@JEWAL	8.75	Pass	9.0	
3	Adekunle Faderera	@Agnes A	8.50	Pass	0.0	
4	Adetola Testimony Oluwafikunayomi	@Adetola Testimony Oluwafikunayomi	8.00	Pass	0.0	

5 rows × 26 columns



In [166... `# Count passes`
`decision_cols = ['Stage1_Decision', 'Stage2_Decision', 'Stage3_Decision', 'Stage4_D`
`new_df['total_passes'] = new_df[decision_cols].apply(lambda x: (x == 'Pass').sum(),`
`new_df['total_fails'] = new_df[decision_cols].apply(lambda x: (x == 'Fail').sum(),`

In [167... `# Create target variable: Did they pass all completed stages?`
`new_df['passed_all_stages'] = (new_df['total_fails'] == 0) & (new_df['completion_ra`

In [168... `# Check min, max, and distribution`
`print("=== average_grade Statistics ===")`

```

print(new_df['average_grade'].describe())

# See unique values or histogram for better insight
print("\n=== Unique Values (Top 20) ===")
print(new_df['average_grade'].value_counts().head(20))

# Optional: Plot histogram to visualize spread
import matplotlib.pyplot as plt
plt.figure(figsize=(10,4))
new_df['average_grade'].hist(bins=20, edgecolor='black')
plt.title('Distribution of average_grade')
plt.xlabel('Average Grade')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()

```

```

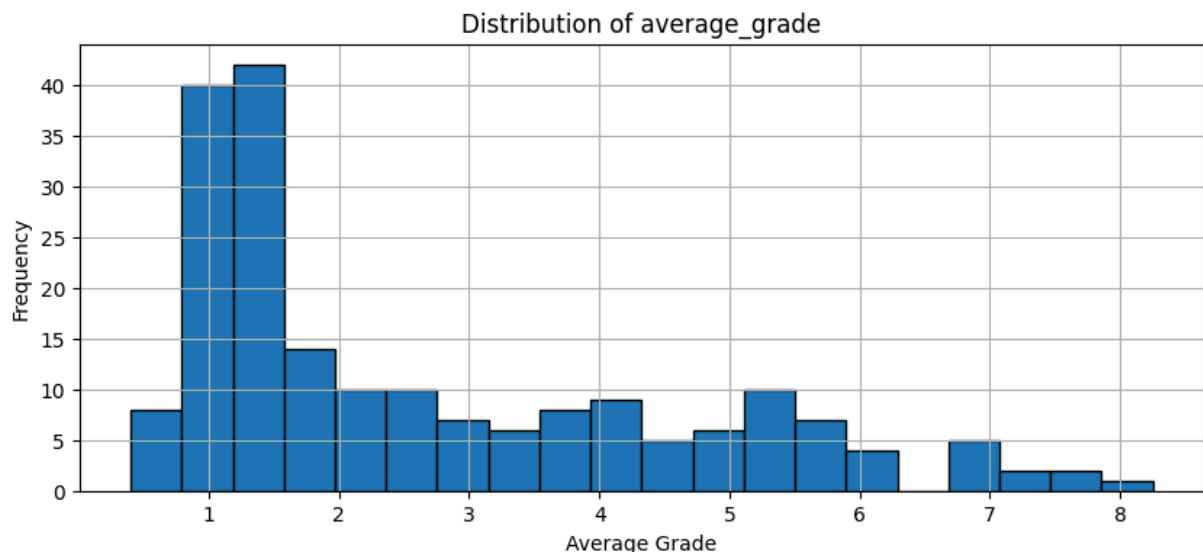
=== average_grade Statistics ===
count      196.000000
mean         2.675867
std          1.902155
min           0.400000
25%          1.200000
50%          1.650000
75%          4.012500
max           8.250000
Name: average_grade, dtype: float64

```

```

=== Unique Values (Top 20) ===
average_grade
1.00      12
1.10      10
1.50       9
1.20       8
1.40       8
1.60       8
1.30       7
2.75       4
3.10       4
1.45       4
5.35       4
0.80       4
1.70       4
0.95       4
2.60       3
3.50       3
1.05       3
0.70       3
0.85       3
1.55       3
Name: count, dtype: int64

```

```
In [171...] # performance category based on average_grade
bins = [0, 2.5, 4.5, 8.25] # Adjust max to match your data
labels = ['Needs Improvement', 'Satisfactory', 'Excellent']

new_df['performance_category'] = pd.cut(
    new_df['average_grade'],
    bins=bins,
    labels=labels,
    include_lowest=True
)
```

```
In [173...] print(f"\nNew features created:")
print(new_df[['Full Name', 'average_grade', 'grade_improvement', 'completion_rate',
              'total_passes', 'performance_category', 'passed_all_stages', ]].head())
```

New features created:

	Full Name	average_grade	grade_improvement
0	Aanuoluwapo Raheem	0.85	-4.25
1	Adedigba Kehinde	3.90	-7.50
2	Adekemi Adetulubo	8.25	-1.25
3	Adekunle Faderera	1.70	-8.50
4	Adetola Testimony Oluwafikunayomi	3.10	-8.00

	completion_rate	total_passes	performance_category	passed_all_stages
0	0.2	0	Needs Improvement	False
1	0.6	2	Satisfactory	True
2	1.0	4	Excellent	True
3	0.2	1	Needs Improvement	True
4	0.4	1	Satisfactory	True

```
In [174...] new_df.to_csv('hng_enhanced_dataset.csv', index=False)
print("\n✓ Enhanced dataset saved to 'hng_enhanced_dataset.csv'")
```

✓ Enhanced dataset saved to 'hng_enhanced_dataset.csv'

```
In [175...] # Machine Learning Model
# Prepare data for modeling - Predict if they'll pass all stages based on all their
X = new_df[['Stage1_Grade', 'Stage2_Grade', 'Stage3_Grade', 'Stage4_Grade', 'Stage7
```

```
'average_grade', 'grade_improvement', 'completion_rate']].copy()
y = new_df['passed_all_stages'].copy()
```

```
In [176... # Remove rows where target is missing
mask = y.notna()
X = X[mask]
y = y[mask]
print(f"\nTraining data shape: {X.shape}")
print(f"Target distribution:\n{y.value_counts()}")
```

Training data shape: (196, 8)

Target distribution:

passed_all_stages

True 134

False 62

Name: count, dtype: int64

```
In [177... # Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

```
In [178... # Train Random Forest model
print("\nTraining Random Forest model...")
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, max_depth=5)
rf_model.fit(X_train, y_train)
```

Training Random Forest model...

Out[178... RandomForestClassifier

► Parameters

```
In [179... # Predictions
y_pred = rf_model.predict(X_test)
y_pred_proba = rf_model.predict_proba(X_test)[:, 1]
```

```
In [180... # Model Evaluation
print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"ROC-AUC Score: {roc_auc_score(y_test, y_pred_proba):.4f}")
```

Accuracy: 0.9500

ROC-AUC Score: 0.9801

```
In [181... print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
False	1.00	0.85	0.92	13
True	0.93	1.00	0.96	27
accuracy			0.95	40
macro avg	0.97	0.92	0.94	40
weighted avg	0.95	0.95	0.95	40

```
In [182... print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Confusion Matrix:

```
[[11  2]
 [ 0 27]]
```

```
In [183... # Feature importance
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\nFeature Importance:")
print(feature_importance)
```

Feature Importance:

	Feature	Importance
5	average_grade	0.227298
3	Stage4_Grade	0.179022
6	grade_improvement	0.159772
0	Stage1_Grade	0.124778
2	Stage3_Grade	0.107906
7	completion_rate	0.084830
4	Stage7_Grade	0.068399
1	Stage2_Grade	0.047996

```
In [184... # Save model
joblib.dump(rf_model, 'hng_success_prediction_model.pkl')
print("\n✓ Model saved to 'hng_success_prediction_model.pkl'")
```

✓ Model saved to 'hng_success_prediction_model.pkl'

```
In [185... # Create predictions dataset
new_df['predicted_success'] = rf_model.predict(X)
new_df['success_probability'] = rf_model.predict_proba(X)[:, 1]
```

```
In [186... # Prediction-based performance
new_df['predicted_performance'] = pd.cut(
    new_df['success_probability'],
    bins=[0, 0.33, 0.66, 1.0],
    labels=['Will go Home', 'Might Finish HNG', 'Will Finish HNG'],
    include_lowest=True
)
```

```
In [188... new_df.to_csv('hng_with_predictions.csv', index=False)
print("✓ Dataset with predictions saved to 'hng_with_predictions.csv'")
```

✓ Dataset with predictions saved to 'hng_with_predictions.csv'

```
In [189... summary = pd.DataFrame({
    'Metric': ['Total Participants', 'Average Grade', 'Pass Rate (%)',
              'Completion Rate (%)', 'High Performers'],
    'Value': [
        len(new_df),
        round(new_df['average_grade'].mean(), 2),
```

```
        round((new_df['passed_all_stages'].sum() / len(new_df)) * 100, 2),  
        round(new_df['completion_rate'].mean() * 100, 2),  
        len(new_df[new_df['performance_category'] == 'Excellent'])  
    ]  
})  
  
print("\nSummary Statistics:")  
print(summary)
```

Summary Statistics:

	Metric	Value
0	Total Participants	196.00
1	Average Grade	2.68
2	Pass Rate (%)	68.37
3	Completion Rate (%)	41.53
4	High Performers	39.00

In []: